Conveyor Line

Scenario A, Round 2 - 60 minutes, teams of 4 to 5

Your task

You build the same three stations as in Round 1, with the same parts and the same wiring. This time the stations form one line. A part is detected first, then sorted, then counted.

Each station may only work on a part that the station before it has handed over. Only one part is on the line at a time.

What the system must do

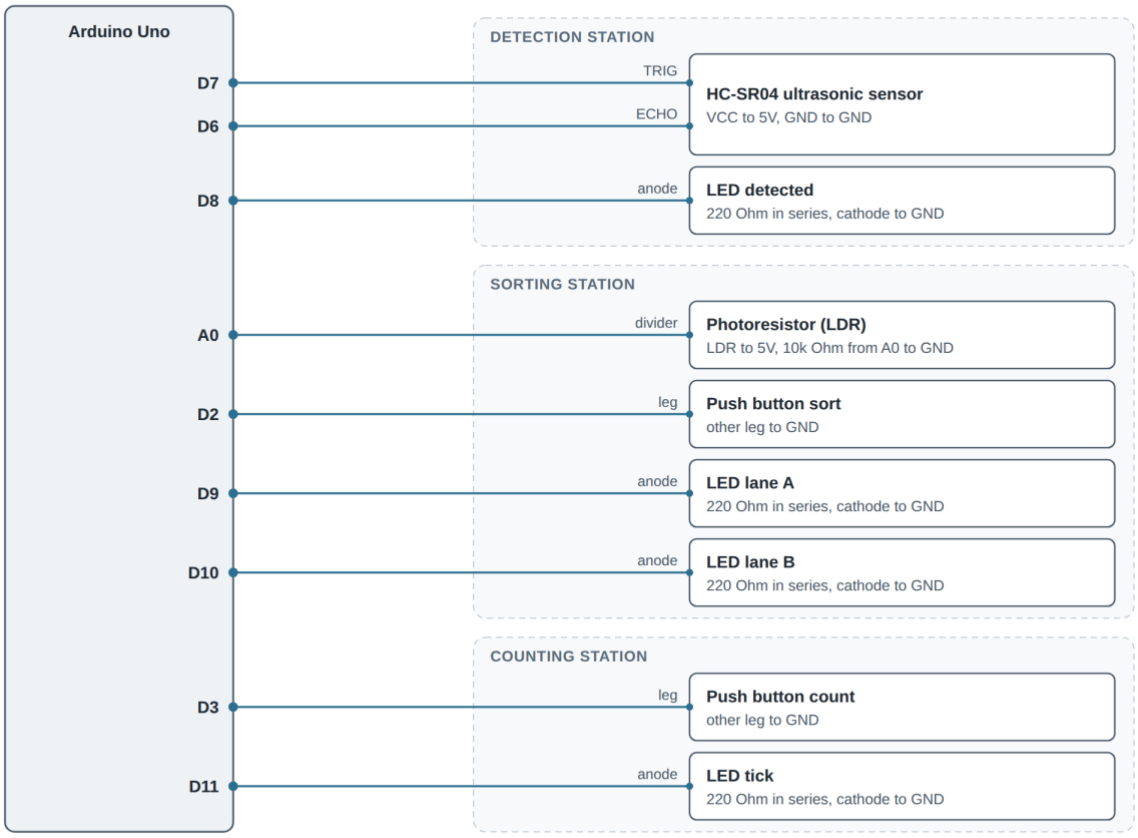
- Detection hands over a new part only when the line is empty.
- Sorting works only on a detected part. Without one it refuses and reports this.
- Counting works only on a sorted part. It reports the lane and the total and then frees the line.
- A part that leaves and comes back while the line is busy is not a new part.

Parts

Part	Quantity
Arduino Uno and breadboard	1
HC-SR04 ultrasonic sensor	1
Photoresistor (LDR)	1
10k Ohm resistor	1
Push button	2
LED	4
220 Ohm resistor	4
Jumper wires	about 20

Wiring

Scenario A Conveyor line (same wiring in both rounds)



The wiring is the same as in Round 1.

The code

You can build the circuit on a real board or in TinkerCad Circuits. In TinkerCad, open the code editor, switch it to Text and paste the code. Fill in every blank marked ___ before you start the program. The code you get is not fully correct. When the system does not do what the list above asks, find the cause and fix it.

Blank	What to fill in
Sorting condition	The stage of a part that sorting waits for.
Counting condition	The stage of a part that counting waits for.
After counting	The stage that makes the line free for the next part.

```
/*
 * Scenario A  Conveyor line
 * Round 2  The stations form one line
 * Fill in every ___ before you upload.
 */

// Pin map (identical in Round 1 and Round 2)
const int PIN_TRIG         = 7;
const int PIN_ECHO         = 6;
const int PIN_LED_DETECTED = 8;
```

```
const int PIN_LDR          = A0;
const int PIN_LED_LANE_A   = 9;
const int PIN_LED_LANE_B   = 10;
const int PIN_BTN_SORT     = 2;
const int PIN_BTN_COUNT    = 3;
const int PIN_LED_TICK     = 11;

const int NEAR_CM        = 15;
const int SORT_THRESHOLD = 500;

// Shared line state. This is the interface between the stations.
// Only one part is on the line at a time.
enum PartStage { NONE, DETECTED, SORTED };
PartStage stage = NONE;
int  lastLane  = 0;      // 1 = lane A, 2 = lane B
int  partCount = 0;
bool nearPrev  = false;
bool sortPrev  = true;
bool countPrev = true;

long readCM() {
  digitalWrite(PIN_TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  long d = pulseIn(PIN_ECHO, HIGH, 20000);
  return (d == 0) ? 999 : d * 0.034 / 2;
}

// Detection station. Hands over DETECTED when a new part arrives
// and the line is empty.
void stationDetection() {
  bool near = readCM() < NEAR_CM;
  digitalWrite(PIN_LED_DETECTED, near ? HIGH : LOW);
  if (near && !nearPrev && stage == NONE) {
    stage = DETECTED;
    Serial.println("Part detected");
  }
  nearPrev = near;
}

// Sorting station. Same operation as in Round 1, but only for a
// detected part. Hands over SORTED.
void stationSorting() {
  bool pressed = digitalRead(PIN_BTN_SORT) == LOW;
  if (pressed && sortPrev) {
    if (stage != ___) {         // sorting waits for this stage
      Serial.println("Sort refused, no detected part");
    } else {
      int v = analogRead(PIN_LDR);
      lastLane = (v < SORT_THRESHOLD) ? 1 : 2;
      digitalWrite(PIN_LED_LANE_A, lastLane == 1 ? HIGH : LOW);
      digitalWrite(PIN_LED_LANE_B, lastLane == 2 ? HIGH : LOW);
      Serial.println(lastLane == 1 ? "Sorted to lane A" : "Sorted to lane B");
      stage = SORTED;
    }
  }
  sortPrev = !pressed;
}

// Counting station. Same operation as in Round 1, but only for a
// sorted part. Frees the line for the next part.
void stationCounting() {
  bool pressed = digitalRead(PIN_BTN_COUNT) == LOW;
  if (pressed && countPrev) {
    if (stage != ___) {         // counting waits for this stage
      Serial.println("Count refused, part not sorted");
    } else {
      partCount++;
      digitalWrite(PIN_LED_TICK, HIGH); delay(80); digitalWrite(PIN_LED_TICK, LOW);
      Serial.print("Lane "); Serial.print(lastLane == 1 ? "A" : "B");
      Serial.print(", total "); Serial.println(partCount);
      digitalWrite(PIN_LED_LANE_A, LOW); digitalWrite(PIN_LED_LANE_B, LOW);
```

```
      stage = ___;               // the line is free again
    }
  }
  countPrev = !pressed;
}

void setup() {
  pinMode(PIN_TRIG, OUTPUT); pinMode(PIN_ECHO, INPUT);
  pinMode(PIN_LED_DETECTED, OUTPUT);
  pinMode(PIN_LED_LANE_A, OUTPUT); pinMode(PIN_LED_LANE_B, OUTPUT);
  pinMode(PIN_BTN_SORT, INPUT_PULLUP);
  pinMode(PIN_BTN_COUNT, INPUT_PULLUP);
  pinMode(PIN_LED_TICK, OUTPUT);
  Serial.begin(9600);
  Serial.println("Conveyor line ready (Round 2)");
}

void loop() {
  stationDetection();
  stationSorting();
  stationCounting();
  delay(50);
}
```

How you work

Every member of the team takes one role.

Role	Responsibility	Works closely with
Line coordinator	Agrees the handover rules with everyone and owns the shared line state in the code. Keeps track of time and signs off the result.	Everyone
Detection developer	Builds the detection station and hands over each new part.	Sorting developer
Sorting developer	Builds the sorting station. Works only on detected parts and hands them on.	Detection and counting developers
Counting developer	Builds the counting station. Counts only sorted parts and frees the line.	Sorting developer
Tester	Tests the full line against the list above and reports what does not work.	Everyone

In a team of four, the line coordinator also takes the role of tester.

Handover rules

Agree these rules before anyone starts building.

Handover	What is passed on and when
Detection to sorting	A new part is handed over once, and only when the line is empty.
Sorting to counting	The sorted part and its lane are handed over once.
Counting to the line	After counting, the line is empty and ready for the next part.

Use the first 10 minutes to agree the handover rules as a team. Keep the last 15 minutes to connect and test the full line.

After the round

Take about 10 minutes as a team and talk about these questions.

- Compared to Round 1, where did you suddenly need to agree on something?
- Did a station act too early or on a part that was not ready?
- When something did not work, how did you find the cause?
- What slowed you down more, technical problems or the handovers?
- Where do handovers like these happen in real companies?